

OvercookedVR – A Cyberphysical Extended Reality Testbed for Multi-modal Human-Robot Teaming

Sushrut Surve*, Jovan Menezes*, Adhyan Prasad and Silvia Ferrari

*Equal contribution

Sibley School of Mechanical and Aerospace Engineering
Cornell University, Ithaca, NY 14850

Abstract—Effective communication is the cornerstone of fluent human-robot teaming. While recent advances in large language models (LLMs) have catalyzed impressive progress in language-based communication, nonverbal channels such as gestures, gaze, and motion, remain comparatively underexplored due to the difficulty of collecting high-quality, multimodal data in the real world. This paper presents OvercookedVR, a cyber-physical testbed built around a virtual-reality (VR) version of Overcooked that provides a controlled environment for measuring teamwork and communication. OvercookedVR enables humans to collaborate with both physical and virtual LLM-enabled robots in shared simulated environments using speech, gesture, gaze, and motion to coordinate cooking tasks. Novel sensor interfaces are developed to record and monitor all these communication channels in real-time. Novel LLM-based AI agent is developed to use these communication channels to infer latent human intent and effectively choose the next best action for simulated and even real robots. The facility also supports collection of inter-human teaming datasets to train models that collaborate with humans as effectively as human partners. Human evaluation studies are conducted in simulation and physical experiments to demonstrate the capabilities of OvercookedVR as a platform for real-time multimodal human-robot teaming.

I. INTRODUCTION

Many emerging robotics applications, ranging from nursing homes to urban search and rescue [5, 10, 3], require robots to partner with humans to achieve shared goals. Compared to homogeneous teams of robots, mixed human-robot teams (MHRT) can potentially improve efficiency and robustness by leveraging complementary skills such as human field experience and domain knowledge [1], and robot data processing and integrated sensor modalities. Progress toward effective MHRTs, however, is impeded by (1) a narrow emphasis on single communication modalities rather than holistic teaming, (2) resource-intensive testing and data collection, and (3) a lack of structured environments for rigorous error detection and evaluation of decision-making models.

Recent advances in natural language processing, particularly large language models (LLMs), have catalyzed new approaches to human-robot interaction [6, 4, 14]. Yet effective human teamwork does not rely solely on language. Humans seamlessly integrate multiple modalities, including body motion, gaze, and facial expression. The ability to interpret such multi-modal cues is especially critical in constrained environments such as underwater or in space, where speech is limited, intermittent, or context-heavy. Although LLMs have

shown early promise in multi-modal understanding, multi-modal human-robot teaming remains underexplored.

Testing and evaluation are key ingredients in the iterative process of developing new technology. However, testing with humans in the loop in open-world settings is not only resource-intensive but also poses serious risks to human safety. Testing in simulation, although an essential tool for robotics research, fails to model the complex decision-making and situational awareness demonstrated by humans. Human-in-the-loop cyberphysical simulation testbeds have emerged as a paradigm for controlled testing of human-robot collaboration algorithms with human-in-the-loop [9].

In addition to control and repeatability, structured task environments are necessary to evaluate reasoning and decision-making. Open-world scenarios exhibit deep task hierarchies, where even simple actions branch into numerous possibilities, complicating evaluation. Games provide an attractive alternative: they offer well-defined team objectives, finite task trees, and precise control over scenario complexity. Such properties have historically advanced AI and deep learning[11, 8, 13]. However, comparable testbeds for human-robot teaming are scarce.

To address these challenges, this paper introduces OvercookedVR (Fig. 1), a cyber-physical testbed designed to study multimodal human-robot teaming with LLMs. This testbed hosts the virtual-reality version of the collaborative game Overcooked, which involves a team of players teaming to prepare recipes and serve as chefs in a restaurant. This testbed integrates recent advances in extended reality, motion capture, and LLMs to provide a collaborative and immersive environment to collect human behavioral data for teaming, monitor implicit modes such as gaze and motion, in addition to explicit modes such as gestures and language, and provide a plug-and-play environment to test various LLMs and agentic AI architectures for human-robot teaming easily. This testbed is specifically designed to make testing easier by allowing testing of AI algorithms with and without robot hardware, to speed up the iterative development process.

II. SYSTEM ARCHITECTURE

The OvercookedVR testbed comprises a human and a robot collaborating to prepare dishes that satisfy the orders. Every order is associated with a delivery time limit. The human-robot team gains points for delivering a dish within the prescribed

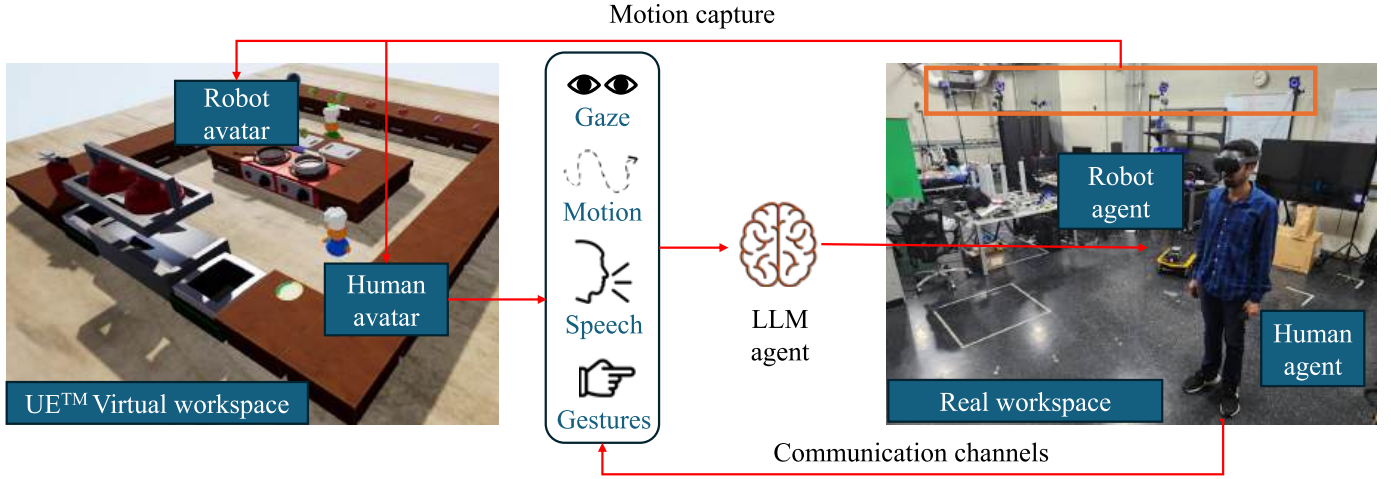


Fig. 1. OvercookedVR is a cyberphysical XR testbed that facilitates teaming between humans and LLM-driven robots using gaze, motion, speech, and gestures. The real-world agents (human agent and robot agent) perceive and move in the real workspace, but collaborate, perceive, and interact in the UE virtual workspace through their virtual counterparts (human avatar and robot avatar).

time limit. Every failed delivery incurs a penalty, and a wrong delivery does not contribute to the overall team score. The goal of the team is to attain the maximum number of points before the total simulation time elapses. The human and the robot embody and control the movement and actions of their respective avatars. Through these avatars, the human-robot team operates and perceives in photorealistic simulation environments designed specifically to test different types of collaboration techniques. This cyber-physical integration of human-robot teams allows the creative freedom to team up in different scenarios created in Unreal Engine, but also allows virtual environments to act as a common medium for safe yet realistic real-time inter-agent and agent-environment interactions. The LLM-based agent running on the robot infers the human intent from different implicit and explicit communication modalities such as motion, gaze, gestures, and speech. This inferred intent is used to decide the next best task for the robot to collaborate effectively with the human. This task is executed by the robot using online planning and control strategies.

A. Simulation environment

The simulation environments in which the agents operate are developed using Unreal Engine (UE) 5. UE was selected as the foundation for the testbed due to its real-time rendering capabilities and high level of photorealism, which provide immersive experiences for human participants and help narrow the gap between simulation and reality for perception-driven tasks. The testbed consists of three different map configurations: *Ring*, *Bottleneck*, *Partition*, and *Quick* [7], as illustrated in Fig. 2. The *Ring* map serves as a baseline environment for evaluating general collaborative behaviors. The *Bottleneck* map introduces spatial constraints that require agents to plan and coordinate paths to avoid collisions in a restricted region. The *Partition* map imposes stricter collaboration requirements by limiting interactions between agents, forcing them to rely



Fig. 2. UE environments corresponding to the (a) *Bottleneck*, (b) *Partition*, and (c) *Ring* configurations.

on each other to complete tasks. As illustrated in Fig. 3, each map consists of six fundamental elements: spawn tables, chopping boards, kitchen tables, cooking pots, trash bins, and delivery tables. Four different spawn table types are implemented: Tomato, Lettuce, Onion, and Plate. A human agent can spawn an ingredient (tomato, lettuce, or onion) or an empty plate by interacting with the corresponding asset mesh on its spawn table, which can then be picked up in VR using the hand controllers. Likewise, an AI agent spawns an item when positioned in front of a spawn table and issued a ‘spawn’ command by the fast mind LLM, where it subsequently collects the spawned item when a ‘pickup’ command is received. Chopping boards are used by both agents to cut freshly spawned ingredients. For humans, this involves placing the ingredient on a chopping board and performing the chopping action by pressing the right controller trigger five times. The AI agent follows a similar process: it first drops the ingredient on the board, then executes the chopping task upon receiving eight ‘chop’ commands from the *fast mind* LLM. During this process, the ingredient transitions through three states, fresh when placed on the board, chopping while the action is underway, and chopped upon completion. Importantly, only fresh ingredients are eligible for chopping.

The kitchen table, whose primary function is to combine ingredients with each other or with a plate, is the most common element across all map configurations. Both human and AI agents perform the combine action by sequentially

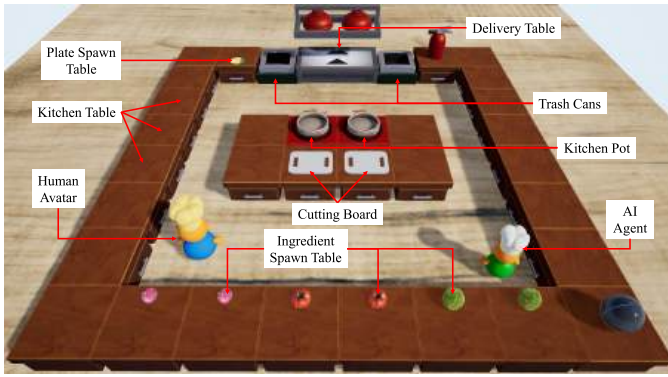


Fig. 3. Virtual Environment in UE showing different elements in the *Ring* map.

placing items onto the table. Once a second compatible item is placed, the assets are merged into a single entity. Only chopped ingredients can be combined, and combinations must involve distinct assets, *e.g.*, two chopped tomatoes or two plates cannot be merged. The kitchen pot is used for cooking tasks, which may involve either a single chopped ingredient or a combined set. Cooking begins when an agent places the ingredient into the pot, initiating a 10-second timer displayed as a progress bar. During this period, the ingredient’s state transitions to cooking. When the timer completes, the cooked ingredient can be transferred onto a plate. The human agent does so by bringing a plate close to the pot, while the AI agent executes the transfer upon receiving a *‘pickup-cooked’* command from the *fast mind* LLM. The plated item can then be delivered. After cooking is complete, a secondary timer begins, visualized through a separate receding progress bar. This timer indicates the interval before the food becomes overcooked and eventually charred. Agents must retrieve the dish before this timer expires; otherwise, the ingredient transitions to a burnt state and is no longer deliverable. Burnt ingredients can still be cleared from the pot with the human agent accomplishing this by intersecting a plate with the pot, while the AI agent performs the same action upon receiving a *‘pickup-charred’* command from the *fast mind* LLM. Once a cooked ingredient is plated, an agent can deliver it by placing the plate on the delivery table. Successful delivery of the correct recipe grants the agent a reward, after which the item is removed from the virtual environment. Conversely, charred ingredients can be discarded by depositing them into the trash bin. In this case, the item is deleted without yielding any reward.

B. Human XR integration

OvercookedVR allows human agents to perceive and interact with the simulation environments using an integration of motion capture and virtual reality (VR) technologies. The human agent wears a Meta Quest Pro VR head-mounted display (HMD) equipped with reflective motion capture markers. Using Optitrack motion capture cameras and their LiveLink package, the human is localized inside the virtual environments. The outside-in tracking bundle from Meta is used to render

frames according to the head and body pose orientation. This integration provides the user with an immersive, interactive first-person experience of the simulation environment (as seen in Fig. 4). The headset is equipped with IR cameras for eye tracking, which enables streaming gaze data. This gaze data, along with motion tracks, offers informative insights into the human’s latent intent. For example, the human agent looks at an object they are deliberating about or moves towards a task they intend to do. Humans use the handheld VR controllers to interact with the environment, such as picking up and placing objects or chopping ingredients. The VR HMD displays the orders on the top right of the FOV of the human, in addition to the progress bars for interactions such as chopping or cooking. The mic mounted inside the headset is used to extract speech commands for the robot. The human agent is also allowed to use the hand controllers to point towards certain locations in the environment to assign tasks to the robot or communicate information to improve the situational awareness of the team. Ray tracing is used to highlight the locations the gestures are meant to point towards. As is common in human-human teams, gestures, gaze, motion, and speech commands are used alongside situational awareness of the environment by the robot agent to infer the intent of the human and choose the next best task to collaborate effectively with the human. Figure 5 presents each human interaction modality as implemented in the developed UE environment.



Fig. 4. Human’s VR avatar interacting with an object in the UE environment.

C. LLM-based agent

In the OvercookedVR testbed, the robot agent employs an LLM-powered AI agent to interpret multi-modal inputs from humans, analyze the current state of the environment, interact with the human in language, and infer the next best action to take to assist the human. The teaming scenario presented by the game is particularly more challenging because of the constant time pressure the team is under. Thus, the AI agent should not only be equipped with strong reasoning abilities, but also be able to use them to infer the next action in real-time. Additionally, the adopted framework should be modular enough to allow for testing with and without VR, or, with and without physical robots, and also facilitate easy testing of different models and reasoning architectures. Hence, this

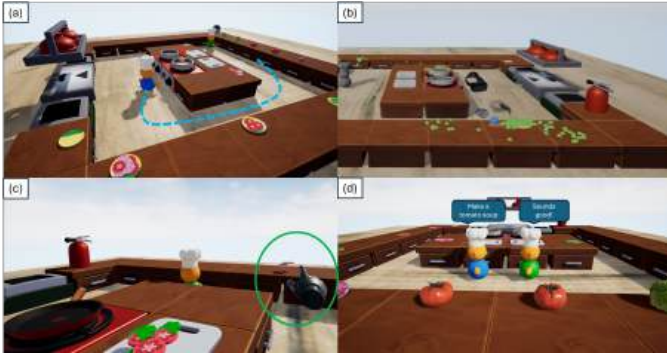


Fig. 5. Different modalities of interaction in UE: (a) motion with track indicated in blue, (b) gaze position at current timestep in blue and past steps in green, (c) gesturing where the human points to an item, and (d) explicitly telling the AI agent what to do through speech.

paper adopts the hierarchical language reasoning framework presented in [7]. Specifically, the AI-agent comprises of 2 sub-modules, namely, *Slow Mind* which focuses on deep reasoning for interactions with the human, and a *Fast Mind* which generates macro-actions to be performed. Both the modules are independent of one another and can be used separately depending on testing and deployment requirements. For instance, the reasoning abilities of larger models can be benchmarked in these environments by using the *Fast mind* only while the inference time of smaller more efficient models can be tested by employing just the *Slow mind*. Fig. 6 shows the flow of information to and the *fast mind* and *slow mind*. The next subsections describe the slow and the *fast mind* in further detail.

1) *Slow mind*: Bidirectional communication between collaborative agents is essential to communicate task-relevant information important for completion. The *Slow mind* ensures that this flow of information is maintained by interpreting multi-modal inputs from the human as commands to be executed by the agent and interacting with the human in speech or text to convey information. Thus, this paper used GPT-4.1 due to its advanced reasoning and interaction abilities. Gaze and motion tracks are implicit modes of communication which are always available as long as the human interacts with the environment unlike explicit modes such as speech or gestures. Thus, unlike in [7] where the *Slow mind* runs only when speech or text command is provided by the human, the *Slow mind* runs continuously to interpret the gaze and motion tracks data.

Interpreting motion data In non-adversarial scenarios such as teaming, the motion of agents can be safely assumed supports the tasks they want to accomplish. Thus, motion tracks can be used to infer the tasks the agents want to accomplish without explicitly conveying intent [2]. As described in Section. II-B, the motion of the human agent is constantly recorded to update the state of the human avatar in the map and stored in the motion history of the human agent. Every time a macro action is completed, the motion start state is set to the current human agent location and the motion track

is reinitialized with the first element being the motion start state. Additionally, reference trajectories are calculated from the motion start state to all possible interactable task locations on the map using A*. Until the next *Slow mind* inference begins, the motion track is continuously recorded. Once the previous *Slow mind* inference cycle is over, the motion track is compared with the reference trajectories using the online waypoint recognition algorithm in [2] to estimate the most probable task the human agent intends to achieve according to the motion data.

Interpreting gaze data Eye tracking is another popular mode of implicit communication which has been used in the teaming literature to infer physiological states of the human and latent intent [12]. Specifically, this paper explores the use of eye gaze tracking to infer the tasks the human is aiming to go to or contemplating about. The UE simulation environment enables accurate tracking of the gaze which can be registered as 3D points in the UE reference frame. Similar to motion track, a gaze track is reinitialized whenever the human agent completes a macro action, under the assumption that the human starts contemplating about the next task to be done after completing one. For every interactable task location, the number of time-steps at which the gaze location was closest to it is recorded. Once the previous *Slow mind* inference cycle is over, the task with the highest count is declared as the most probable task the human intends to achieve according to gaze data.

Interpreting gesture data Gestures are used by human agents to convey commands to the robot agent. Gestures when used with speech can greatly decrease ambiguity introduced with just speech commands and ground the commands in context very robustly. Unlike motion and gaze data, gesture data is only recorded when the human presses the left trigger to point at a specific location on the map. The ray traced gesture location is used to alongside the speech data recorded from the HMD mic to process commands for the robot agent. It is important to note that gestures, can be accompanied without any speech, which is also an important interaction modality when the human agent chooses not to talk under the effect of time pressure levied by the task.

Multi-modal human input reasoning After processing all the modalities, a list comprising the most probable tasks the human intends to do according to gaze and motion, the most probable task the human commands using gestures, and speech is provided to the *Slow mind* LLM. Using this list alongside command history and environment state, the LLM interprets the concrete intentions of the human partner. For instance, ... (implicit for conflict avoidance and explicit for task completion) One important thing to note is that the LLM is input length invariant, which means that the LLM is capable of using any combination of the modality available at a time instant to reason about the next action for the robot. This is especially true in real-world teaming scenarios when, depending on the circumstance, the human may arbitrarily choose a combination of supported inputs to convey commands. The prompt combining this with the prompt used in [7] is provided

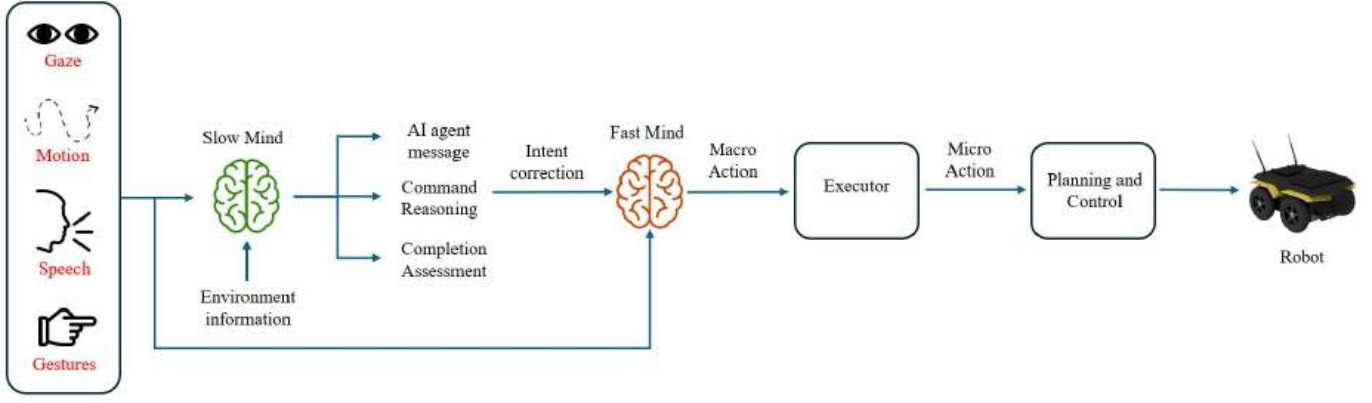


Fig. 6. Multi-modal inputs (gaze, motion, speech, gestures) from the human are used by the *slow mind* to infer commands and latent intent of the human. The *slow mind* is also responsible for the completion assessment of a commanded action and facilitating bidirectional communication with the user through speech. The *fast mind* leverages the intent predicted from the *slow mind* to select the next macro-action. The planner and low-level controller use this action to decide the actions to be taken by the robot at each time step.

in the Appendix.

Interaction with the human In addition to processing multi-modal input data from the human, the *Slow mind* is also responsible for interacting directly with the human and providing completion assessment for tasks commanded by the human agent. Every time the human commands the robot with explicit communication modes, such as speech or gestures, to complete a task, it gets added to the commanded task queue. The completion assessment ensures that the tasks in this queue are completed. If there are no tasks in the queue, this assessment is avoided. Additionally, every time the *Slow mind* runs, it uses the environment state, action history, gaze tracks, and motion tracks to infer the chat summary of the tasks the robot is trying to achieve. If a completion assessment is being performed, information from this assessment is also added to the summary.

2) *Fast mind*: Once the *Slow mind* has inferred the command from multi-modal inputs from the human, the *Fast mind* is responsible for inferring the next best macro action in the macro action set to be completed by the robot agent. An advantage of defining this human-robot teaming in a game setting, such as this, is that there is a finite-sized macro action set to choose the next action from. Additionally, the *fast mind* is expected to run at higher frequencies than the *slow mind* to demonstrate real-time reaction to human actions. Thus, the lightweight LLM Llama-13B-chat is chosen to have low inference latency. Conditioned on the current environment state, multi-modal inputs, and the human command, the LLM is employed to predict the probability of all actions in the macro-action set. Finally, the most probable macro action is chosen as the output of *fast mind*. Since the *fast mind* runs at a higher frequency than the *Slow mind*, there are two more cases that are encountered in real-time implementation that need to be tackled. The first case is when the *Slow mind* is still working on inferring the human command, while the *fast mind* is done inferring the last macro-action. In this case, the multi-modal inputs are directly used to infer the macro-action.

The light-weight LLM might not perform as well as the *Slow mind* LLM in this case. However, whenever the *Slow mind* is done inferring the human command, the prompt fed to the *fast mind* is conditioned, and the next macro-action selection procedure is further informed to improve performance. The second case is when there is explicit (speech and gesture) from the human, in which case the *fast mind* infers the next best macro-action based on the chat summary provided by the *Slow mind* and the implicit inputs (gaze and motion).

D. Planning and Control

Once reasoning is finished from the fast/slow mind, the process transitions to an executor module which hierarchically structures the agent’s behavior into high-, mid-, and low-level task abstractions.

In-game task execution High-level (macro) actions correspond to generic objectives, such as chopping ingredients, cooking soups, and plating and delivering completed meals. Mid-level tasks operationalize these objectives by formalizing staged execution phases (e.g. seek, approach, interact, judge), and explicit success or failure outcomes. Therefore, these actions provide the bridge between symbolic goals and executable behavior. At the lowest level, utility functions enable action execution by planning feasible navigation paths in the grid world. In particular, a breadth-first search (BFS) algorithm is employed to identify the shortest path between a start location and then next target location. The search expands outward from the start position in discrete (grid) steps until the goal is reached, guaranteeing that the first path discovered is the shortest in terms of move counts. The executor then interprets the path by issuing incremental movements to move to the first node in the path from the agent’s current location. Additional logic is used to re-plan the path when obstructed by other agents or where there is no path that exists.

Robot agent control Under the assumptions that the physical environment is static and contains no obstacles other than the VR equipped human agent, the base control station simply

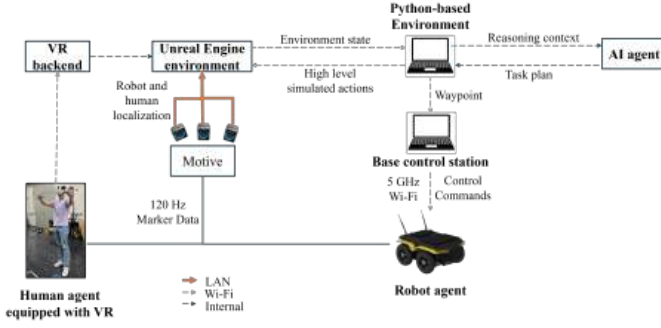


Fig. 7. Fully integrated system communication; data transfer over LAN, Wi-Fi, and Internal is shown in orange, grey, and blue respectively

intercepts the interpreted grid location to move to from the LLM-based agent’s mid level task planner. From here, the control station applies the required coordinate transformation to the aforementioned location to retrieve the global coordinates (as defined by the Motive outside-in tracking) of the point to travel to. These coordinates are then compared to against the robot’s current position and orientation in the same reference frame. Using a feedback linearization controller, the system computes forward and angular velocity commands that drive the robot to the desired location. The robot used in these experiments is the Clearpath Jackal UGV, which operates under a four-wheeled differential-drive dynamics model. The ROS application applies these commands to the physical platform, enabling real-time execution of motion in the environment. To ensure safe operation, the computed velocities are constrained by predefined limits, and motion is terminated once a proximity threshold to the goal has been reached.

E. System Communication

As shown in Fig.7, all of the aforementioned components are integrated into a unified system architecture that enables interaction between the physical and virtual agents. Both agents are localized within a common reference frame defined by the Motive motion capture streaming system. Their respective motions are then mirrored within the UE environment. At each update timestep, the UE environment reads the state of all dynamic actors in the simulation. For each virtual object—whether an individual ingredient, a plate, or a combination—the system records its unique instance name, spatial location, semantic state (e.g., fresh, chopped, cooked, etc.), and whether the object is currently being held by an agent. This information is organized into a data structure and transmitted via user datagram protocol (UDP) to the Python-based environment, where it is parsed, stored, and used to update local visualizations. In this way, the same information both provides contextual grounding for the AI agent’s next inference and guarantees that the Python environment accurately mirrors the evolving simulation state. The AI agent’s macro-level actions are similarly communicated over UDP, with routing determined by the nature of the action. As described in Section II-D, movement actions are forwarded to the base

control station and subsequently to the physical robot via ROS, whereas task-level actions such as cooking or delivering are executed within the UE simulation (see Section 4.1). When the robot executes physical movement in the real environment, its motion is tracked by Motive alongside the human’s motion, thereby closing the system feedback loop and maintaining synchronization across agents and environments. Because the system must balance computational demands with the need for the human and robot to operate in a cable-free workspace, data communication is distributed across both Ethernet/LAN and Wi-Fi networks. Although Motive streams motion data at a fixed rate of 120 Hz, the motion and gaze information are frequency-normalized within UE so that the Python environment receives temporally consistent updates at each timestep. This normalization step is critical to maintaining accuracy and synchronization across the system.

III. EXPERIMENTS

This section aims to demonstrate some of the functionalities and capabilities of the OvercookedVR testbed. The first experiment demonstrates how implicit communication modes can be used to predict latent human intent in teaming scenarios, and the second experiment provides a complete run-through of a human-robot teaming experiment conducted with the HLA agentic LLM architecture running on the robot agent.

A. Gaze/motion data extraction

As described in Section. II-C, gaze and motion tracks can be used to infer the task the human is contemplating or about to do. This information is important for the robot agent to avoid assuming tasks the human agent is trying to do, as task conflicts like these can decrease the team’s efficiency. Additionally, this information can also help the robot infer the tasks that can be parallelized or can aid the human in their current task. In this experiment, a human participant is shown 3 possible task locations, denoted by w_1 , w_2 , and w_3 in the OvercookedVR environment map. The human participant arbitrarily chooses one of the tasks and moves towards it. The chosen task is not declared to the participant. The goal of this experiment is to show that gaze and motion tracks can be used by the robot to infer this desired task. The gaze track obtained as a 3D point at every time step is first projected on the X-Y plane to get a 2D gaze track projection. The probability of the user observing a specific task location is evaluated by centering a Gaussian at the 2D gaze track projection and evaluating the PDF at the center of every grid cell with covariance equal to the gaze tracking measurement covariance obtained from the VR headset calibration procedure. These weights obtained for every grid cell are normalized by dividing by the sum of weights of all the grid cells across the grid, to obtain the probability of a specific task location being observed by the human. As the experiment started, the human participant scanned the entire environment, and hence the probabilities are distributed almost equally across the grid. At 7 seconds, as the human participant fixes their attention on w_1 , the probabilities start to concentrate near the grid cells occupied by w_1 as

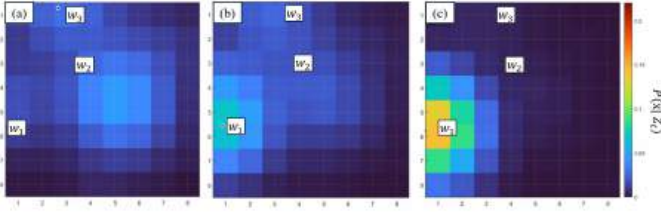


Fig. 8. The probability of the desired task location given the gaze track measurements at (a) $t = 0$ seconds, (b) $t = 7$ seconds, and (c) $t = 15$ secs

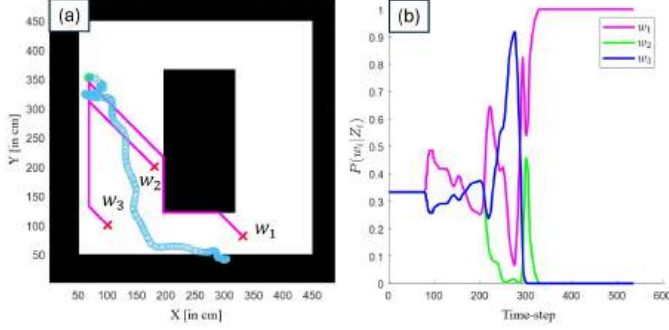


Fig. 9. (a) The reference trajectories in magenta used by the online waypoint algorithm and the observed trajectory in light blue. (b) The probability of the desired task according to motion tracks plotted against time-steps

shown in Fig. 8(b). Finally, at the end of the 15 seconds, when the human reaches w_1 and starts doing their task, it can be seen that the probabilities peak at the task location. Thus, it can be observed that the probabilities start converging at the desired task location much earlier than the human participant reaches that location. Thus, gaze can be used to predict the latent desired task of the human.

The motion tracks of the human participant are also recorded during these experiments in the Unreal Engine reference frame. Using the online waypoint recognition algorithm in [2], the probability of a task being the desired task for the human given a history of observed human trajectory is estimated by comparing the observed trajectory with reference trajectories to the waypoints provided by a planner. In this experiment, A^* is used to calculate the reference trajectories joining the starting point, denoted as a green dot, to w_1, w_2 , and w_3 illustrated by a red cross in Fig.9(a). Fig. 9(b) plots the evolution of the probability of the desired task location predicted for each waypoint. For the first 200 time-steps, the trajectory is non-distinctive and hence the algorithm evaluates equal probabilities for all 3 locations. As time progresses, this probability quickly starts diverging to 1 for w_1 . More importantly, this occurs well before the agent reaches w_1 at time-step 500. Thus, the motion tracks can also be analyzed online to predict a human's intended task and location.

B. Collaboration experiments

To demonstrate the collaborative capabilities of the OvercookedVR testbed, a human-robot teaming experiment is conducted in which a human participant collaborates with an

AI agent embodied in a real-world mobile robot to prepare and deliver a two-ingredient recipe consisting of tomato and lettuce. The sequence of interactions for this experiment is illustrated in Fig. 10, with the top row depicting the human and the AI agent in the virtual environment and the bottom row showing the corresponding real-world execution by the robot and the human. At the start of the task, the virtual avatar of the human and AI agent are positioned near their respective spawn locations in the UE map (subfigures a.i, b.i). The AI agent spawns a fresh lettuce and immediately picks it up from the lettuce spawn table. Meanwhile, the human agent prepares to spawn a tomato. Each agent then navigates the environment independently to perform their respective sub-tasks. The AI agent moves to a chopping board and begins processing the lettuce, while the human agent spawns and chops a tomato on another board. This phase highlights parallel task execution, enabled by implicit coordination and shared situational awareness. Once both ingredients are chopped, the human positions themselves near the AI agent to enable the combination step (subfigures a.ii, b.ii). This coordinated maneuvering highlights how agents operate in parallel yet synchronize at critical points in the workflow.

After combining the ingredients, task responsibilities diverge again. The AI agent proceeds to the plate spawn table to retrieve a clean plate, while the human agent transfers the combined ingredients into the kitchen pot to initiate the cooking process. The ongoing cooking task is visualized through a progress bar as shown in subfigures (a.iii, b.iii). When the cooking timer completes, the AI agent retrieves the plated dish by executing a *'pickup-cooked'* action and carries it to the delivery table. At the same time, the human agent initiates the next recipe cycle, demonstrating how task parallelism can improve throughput (subfigures a.iv, b.iv). This experiment demonstrates the fluid division of labor and synchronization mechanisms that OvercookedVR enables. By combining human intuition with the LLM driven AI agent's robotic embodiment, the system supports not only sequential task sharing but also parallel execution, both of which are essential for effective human-robot collaboration in cyber-physical environments and real-world settings.

IV. CONCLUSION

This study presents OvercookedVR, a cyber-physical testbed designed to investigate multi-modal human-robot collaboration. By adapting a cooperative cooking game to a virtual reality setting and integrating LLM-powered AI agents with physical robots, the platform enables systematic study of teamwork in scenarios requiring coordination, communication, and joint action. The testbed supports both implicit communication cues, such as motion and gaze, and explicit cues, including speech and gestures, providing rich multimodal inputs to LLM-based agents. Through experiments that combine virtual and physical interactions, the results demonstrate how humans and robots can effectively collaborate to complete shared tasks. These findings highlight the potential of OvercookedVR to bridge the gap between simulated collaboration

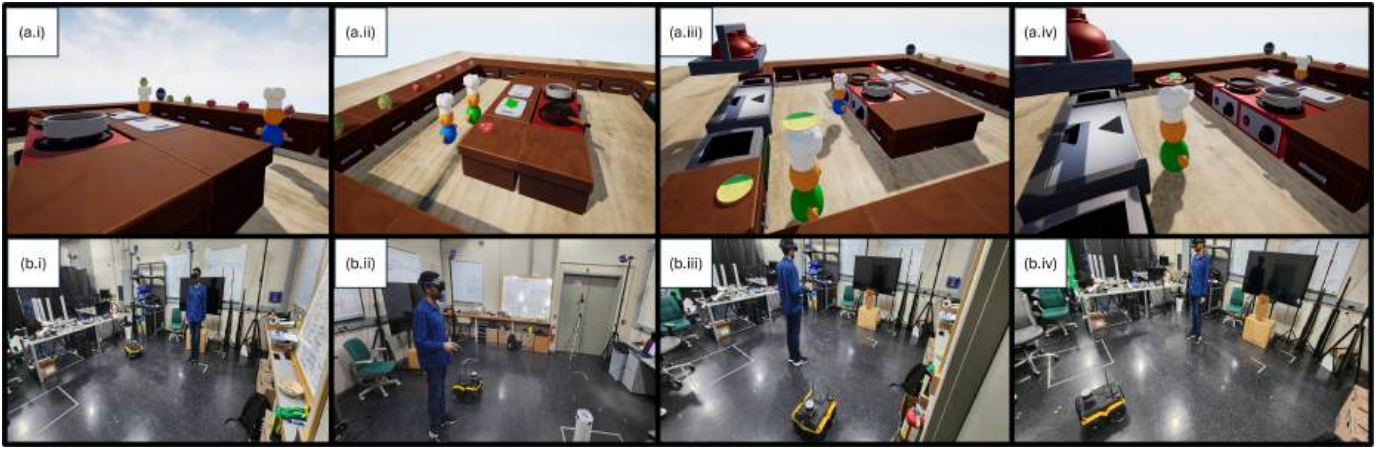


Fig. 10. Image frames displaying various time points in human-robot collaboration experiment. The top row features frames from the virtual environment showing the human avatar and the AI agent, while the bottom row shows the human and robot interacting in the real world.

and real-world deployment, offering a more comprehensive framework for evaluating human–robot teaming dynamics. The platform also serves as a valuable tool for developing and benchmarking algorithms for human–robot collaboration that leverage multimodal communication. Future work will explore how embedding these inputs can further improve LLM-based decision-making for robot planning and control.

REFERENCES

- [1] Mary L Cummings, Jonathan P How, Andrew Whitten, and Olivier Toupet. The impact of human–automation collaboration in decentralized multiple unmanned vehicle control. *Proceedings of the IEEE*, 100(3):660–671, 2011.
- [2] Jia Guo, Sushrut Surve, Zilong He, Silvia Ferrari, and Sarah Keren. Online waypoint recognition of controlled agents in uncertain environments. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12600–12606, 2025. doi: 10.1109/ICRA55743.2025.11128799.
- [3] Jonathan P How, Cameron Fraser, Karl C Kulling, Luca F Bertuccelli, Olivier Toupet, Luc Brunet, Abe Bachrach, and Nicholas Roy. Increasing autonomy of uavs. *IEEE Robotics & Automation Magazine*, 16(2):43–51, 2009.
- [4] Avnish Kanungo. HUMAN IN THE LOOP: AN APPROACH TO OPTIMIZE LLM BASED ROBOT TASK PLANNING. 4 2025. doi: 10.25394/PGS.28828253.v1. URL https://hammer.purdue.edu/articles/thesis/_b_HUMAN_IN_THE_LOOP_AN_APPROACH_TO_OPTIMIZE_LLM_BASED_ROBOT_TASK_PLANNING_b_/28828253.
- [5] Hema S Koppula, Ashesh Jain, and Ashutosh Saxena. Anticipatory planning for human-robot teams. In *Experimental robotics*, pages 453–470. Springer, 2016.
- [6] Younes Lakhnati, Max Pascher, and Jens Gerken. Exploring a gpt-based large language model for variable autonomy in a vr-based human-robot teaming simulation. *Frontiers in Robotics and AI*, 11:1347538, 2024.
- [7] Jijia Liu, Chao Yu, Jiaxuan Gao, Yuqing Xie, Qingmin Liao, Yi Wu, and Yu Wang. Llm-powered hierarchical language agent for real-time human-ai coordination. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems*, pages 1219–1228, 2024.
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [9] Andre Paradise, Sushrut Surve, Jovan C. Menezes, Madhav Gupta, Vaibhav Bisht, Kyung Rak Jang, Cong Liu, Suming Qiu, Junyi Dong, Jane Shin, and Silvia Ferrari. Realthasc—a cyber-physical xr testbed for ai-supported real-time human autonomous systems collaborations. *Frontiers in Virtual Reality*, Volume 4 - 2023, 2023. ISSN 2673-4192. doi: 10.3389/frvir.2023.1210211. URL <https://www.frontiersin.org/journals/virtual-reality/articles/10.3389/frvir.2023.1210211>.
- [10] Jorge Pena Queralta, Jussi Taipalmaa, Bilge Can Pullinen, Victor Kathan Sarker, Tuan Nguyen Gia, Hannu Tenhunen, Moncef Gabbouj, Jenni Raitoharju, and Tomi Westerlund. Collaborative multi-robot search and rescue: Planning, coordination, perception, and active vision. *Ieee Access*, 8:191617–191643, 2020.
- [11] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.
- [12] Sushrut Surve, Jia Guo, Jovan C. Menezes, Connor Tate, Yiting Jin, Justin Walker, and Silvia Ferrari. Unrealthasc – a cyber-physical xr testbed for underwater real-time human autonomous systems collaboration. In *2024 33rd IEEE International Conference on Robot and Human Interactive Communication (ROMAN)*, pages 196–203, 2024. doi: 10.1109/RO-MAN60168.2024.10731209.

- [13] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782): 350–354, 2019.
- [14] Shao Zhang, Xihuai Wang, Wenhao Zhang, Yongshan Chen, Landi Gao, Dakuo Wang, Weinan Zhang, Xinbing Wang, and Ying Wen. Mutual theory of mind in human-ai collaboration: An empirical study with llm-driven ai agents in a real-time shared workspace task. *arXiv preprint arXiv:2409.08811*, 2024.